

1. Tree

1.1. Number of prime numbers $\leq n$

Complexity: $O(n^{2/3})$ (don't include `#define int ll`) · Tags: Number of prime numbers $\leq n$, Used when $n \leq 1e12$

```
const int N = 2e7 + 10, nPrimes = 1270608, phiN = 100000, phiK = 100;
int pc, prm[nPrimes], pref[N], dpPhi[phiN][phiK];

auto auto_ = [] {
    static uint64_t f[(N >> 6) + 1]{};
    for(int i = 2; i < N; i++) {
        if(!(f[i >> 6] >> (i & 63) & 1)) prm[pc++] = i;
        pref[i] = pc;
        for(int j = 0, k; (k = i * prm[j]) < N; j++) {
            f[k >> 6] |= 1ULL << (k & 63);
            if(i % prm[j] == 0) break;
        }
    }
    for(int n = 0; n < phiN; n++) dpPhi[n][0] = n;
    for(int n = 0; n < phiN; n++) {
        for(int k = 1; k < phiK; k++) {
            dpPhi[n][k] = dpPhi[n][k - 1] - dpPhi[n / prm[k - 1]][k - 1];
        }
    }
    return 0;
}();

// Legendre's Phi Function
// Number of positive integers  $\leq n$  that are not divisible by any of the first k primes
ll lPhi(ll n, int k) {
    if(n < phiN && k < phiK) return dpPhi[n][k];
    if(k == 1) return (n + 1) >> 1;
    if(prm[k - 1] >= n) return 1;
    return lPhi(n, k - 1) - lPhi(n / prm[k - 1], k - 1);
}

// Number of prime numbers  $\leq n$ , Used when  $n \leq 1e12$ 
ll lehmer(ll n) {
    if(n < N) return pref[n];
    ll w, res;
    int b = int(sqrtl(n)), c = int(lehmer( ll(cbrtl(n)) )), a =
int(lehmer( ll(sqrtl(b)) ));
    b = int(lehmer(b));
    res = lPhi(n, a) + ((b + a - 2LL) * (b - a + 1) >> 1);
    for (int i = a, j, lim; i < b; i++) {
        w = n / prm[i];
        lim = int(lehmer( int(sqrtl(w)) ));
        res -= lehmer(w);
        if(i <= c) for(j = i; j < lim; j++) res += j - lehmer(w / prm[j]);
    }
    return res;
}
```

1.2. Fast Divisors List

Complexity: $n \log n$ · Tags: math

```
// sumD =  $\sum \text{floor}((N - 1) / i)$ ,  $i = 1..N-1$ 
const int N = 1e6 + 1, sumD = 13970034;
int cnt[N], start[N], all[sumD];
auto pre_compute = [] {
    for (int i = 1; i < N; ++i)
        for (int j = i; j < N; j += i)
            cnt[j]++;
    for (int i = 1; i < N; ++i)
        start[i] = start[i - 1] + cnt[i], cnt[i] = 0;
    for (int i = 1; i < N; ++i)
        for (int j = i; j < N; j += i)
            all[start[j - 1] + cnt[j]++] = i;
    // divisors of x
    // L = start[x - 1]
    // R = start[x]
    // for(int i = L; i < R; i++) d = all[i]
    return 0;
}();
```

1.3. Ternary Search

Complexity: $O(\log R)$

```
ld ternary_search(ld l, ld r) {
    for (int i = 0; i < 300; i++) {
        ld m1 = l + (r - l) / 3;
        ld m2 = r - (r - l) / 3;
        ld f1 = f(m1);
        ld f2 = f(m2);
        // change here to <
        if (f1 > f2) l = m1;
        else r = m2;
    }
    return l;
}
```